

Task 7: Session 7 (AIML) - Assignment Questions

Name: Mihir Sopan Patil

Domain: AIML

College & Branch Name: Zeal Polytechnic, Diploma In AIML

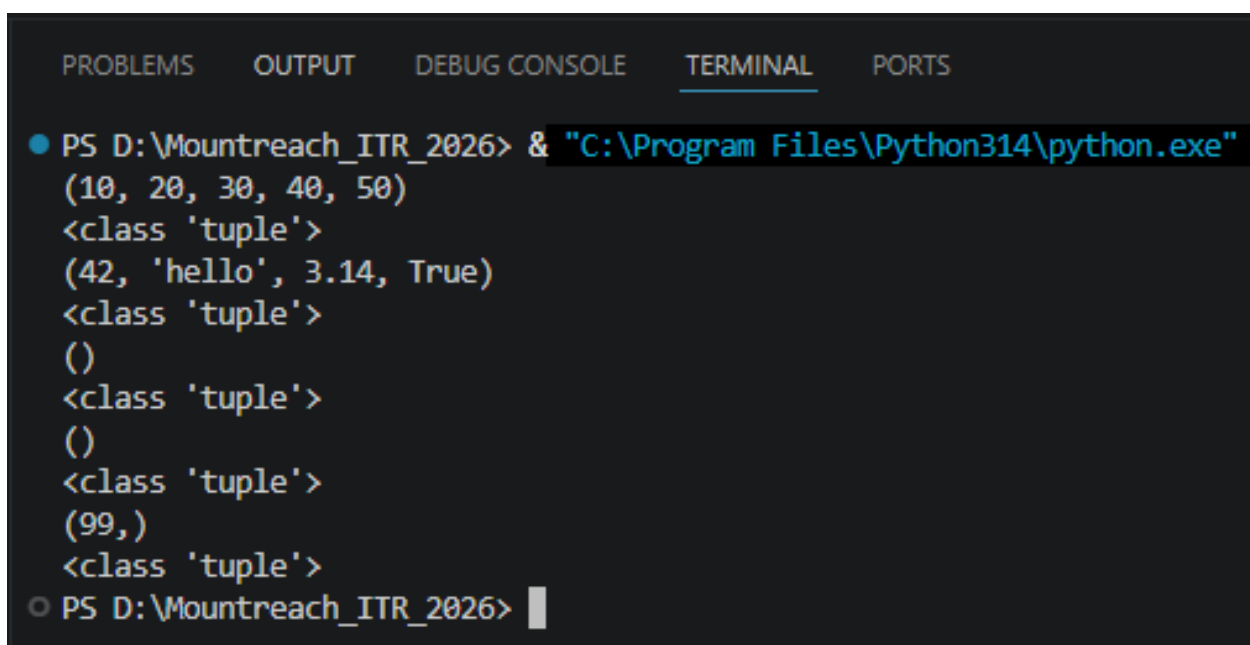
1. Creating Tuples

Write a program to create tuples using different methods:

- A tuple with 5 numbers using parentheses.
- A mixed tuple containing integer, string, float, and boolean.
- An empty tuple using two different ways.
- A single-element tuple with the value 99.

Print all tuples and their types. Add comments explaining the rules for single-element tuples.

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
(10, 20, 30, 40, 50)
<class 'tuple'>
(42, 'hello', 3.14, True)
<class 'tuple'>
()
<class 'tuple'>
()
<class 'tuple'>
(99,)
<class 'tuple'>
○ PS D:\Mountreach_ITR_2026> |
```

Program Code:

```
# Tuple With 5 Numbers Using Parentheses
t1 = (10, 20, 30, 40, 50)
print(t1)
print(type(t1))

# Mixed Tuple
t2 = (42, "hello", 3.14, True)
print(t2)
print(type(t2))

# Empty Tuple - Two Ways
t3 = ()
t4 = tuple()
print(t3)
print(type(t3))
print(t4)
print(type(t4))

# Single Element Tuple
# Rule: A Comma Is Required To Make It A Tuple.
# (99) Is Just An Int; (99,) Is A Tuple.
t5 = (99,)
print(t5)
print(type(t5))
```

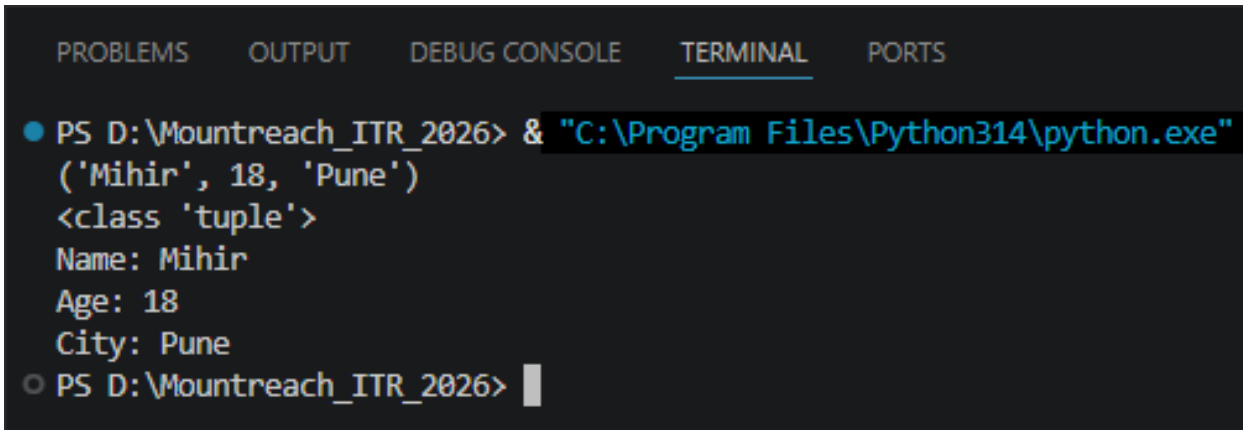
2. Tuple Packing

Create a tuple without using parentheses (tuple packing) to store: your name, age, and city.

Print the tuple and its type.

Then unpack the tuple into three separate variables and print them individually.

Output:



```
● PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
('Mihir', 18, 'Pune')
<class 'tuple'>
Name: Mihir
Age: 18
City: Pune
○ PS D:\Mountreach_ITR_2026> █
```

Program Code:

```
# Tuple Packing - No () Needed
person = "Mihir", 18, "Pune"
print(person)
print(type(person))

# Unpacking Into Separate Variables
name, age, city = person
print("Name:", name)
print("Age:", age)
print("City:", city)
```

3. Indexing & Negative Indexing

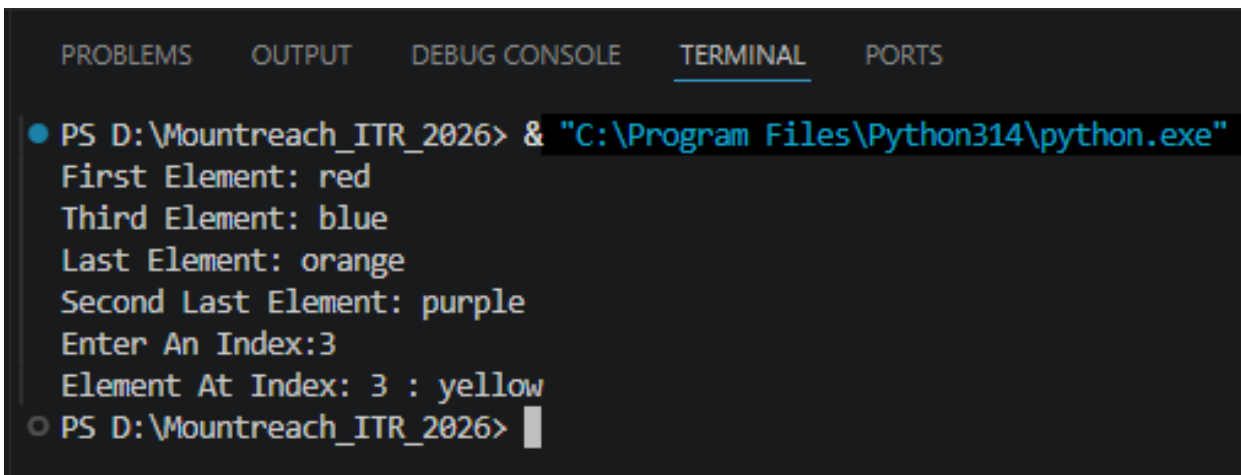
Create the tuple: colors = ('red', 'green', 'blue', 'yellow', 'purple', 'orange')

Print:

- First element
- Third element
- Last element (using negative index)
- Second last element (using negative index)

Take an index number as input from the user and print the element at that index.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
First Element: red
Third Element: blue
Last Element: orange
Second Last Element: purple
Enter An Index:3
Element At Index: 3 : yellow
○ PS D:\Mountreach_ITR_2026>
```

Program Code:

```
colors = ('red', 'green', 'blue', 'yellow', 'purple', 'orange')

print("First Element:", colors[0])
print("Third Element:", colors[2])
print("Last Element:", colors[-1])
print("Second Last Element:", colors[-2])

i = int(input("Enter An Index:"))
print("Element At Index:", i, ":", colors[i])
```

4. Slicing Tuples

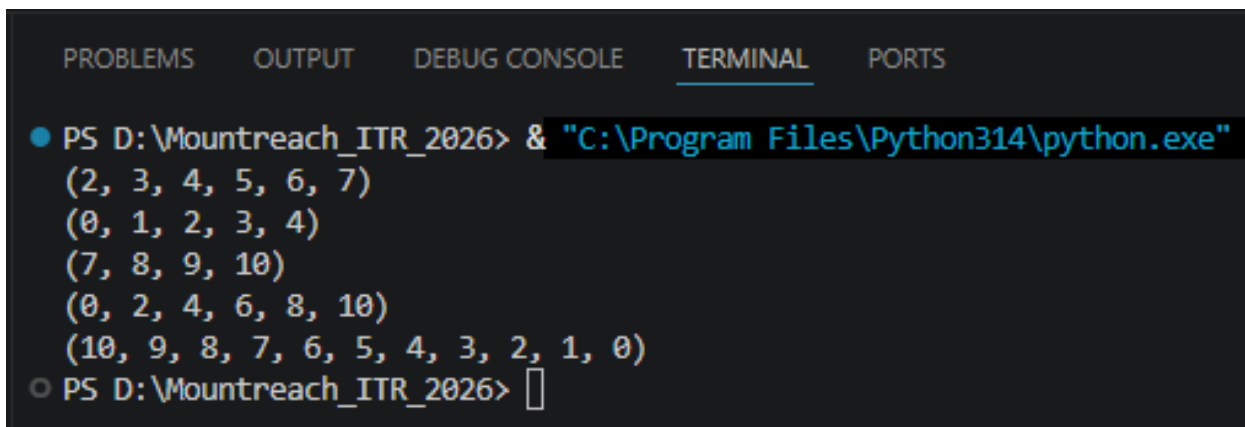
Given the tuple: numbers = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

Using slicing, print:

- Elements from index 2 to 7
- First 5 elements
- Last 4 elements
- Every second element
- The entire tuple in reverse order

Add comments explaining the slicing syntax.

Output:



```
PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
(2, 3, 4, 5, 6, 7)
(0, 1, 2, 3, 4)
(7, 8, 9, 10)
(0, 2, 4, 6, 8, 10)
(10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0)
PS D:\Mountreach_ITR_2026>
```

Program Code:

```
numbers = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

# Syntax: tuple[start:stop:step]
# stop Index Is Excluded

# Index 2 To 7 (stop=8 To Include 7)
print(numbers[2:8])

# First 5 Elements (Index 0 To 4)
print(numbers[:5])

# Last 4 Elements
print(numbers[-4:])
```

```
# Every Second Element (step=2)
print(numbers[::2])

# Reverse (step=-1)
print(numbers[::-1])
```

5. Nested Tuples

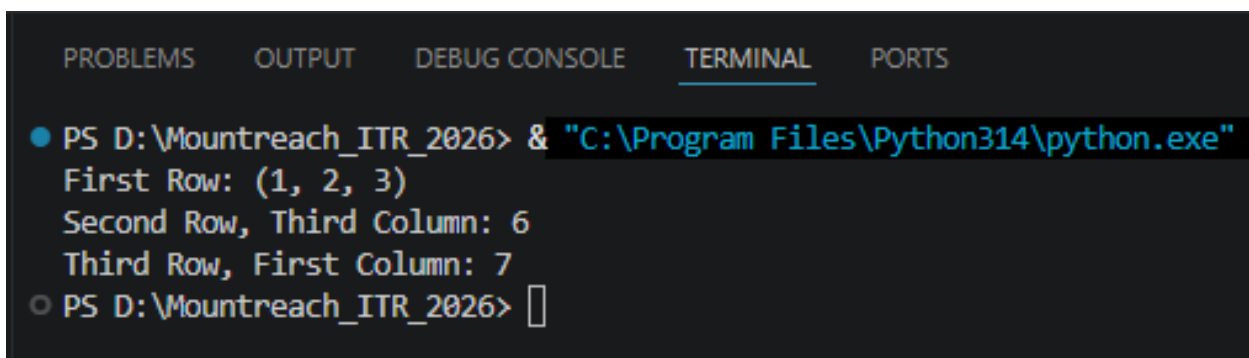
Create a nested tuple to represent a 3•3 matrix:

```
matrix = ((1, 2, 3), (4, 5, 6), (7, 8, 9))
```

Print:

- The first row
- The element at second row, third column
- The element at third row, first column

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
First Row: (1, 2, 3)
Second Row, Third Column: 6
Third Row, First Column: 7
○ PS D:\Mountreach_ITR_2026> █
```

Program Code:

```
matrix = ((1, 2, 3), (4, 5, 6), (7, 8, 9))

print("First Row:", matrix[0])
print("Second Row, Third Column:", matrix[1][2]) # 6
print("Third Row, First Column:", matrix[2][0]) # 7
```

6. Iterating & Unpacking

Create a tuple: `student = ('Rahul', 20, 'Computer Science', 'A')`

- Iterate through the tuple using a for loop and print each item.
- Unpack the tuple into four variables (name, age, branch, grade) and print them with descriptive messages.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
Rahul
20
Computer Science
A
Name: Rahul
Age: 20
Branch: Computer Science
Grade: A
○ PS D:\Mountreach_ITR_2026> █
```

Program Code:

```
student = ('Rahul', 20, 'Computer Science', 'A')

# Iterate With for Loop
for item in student:
    print(item)

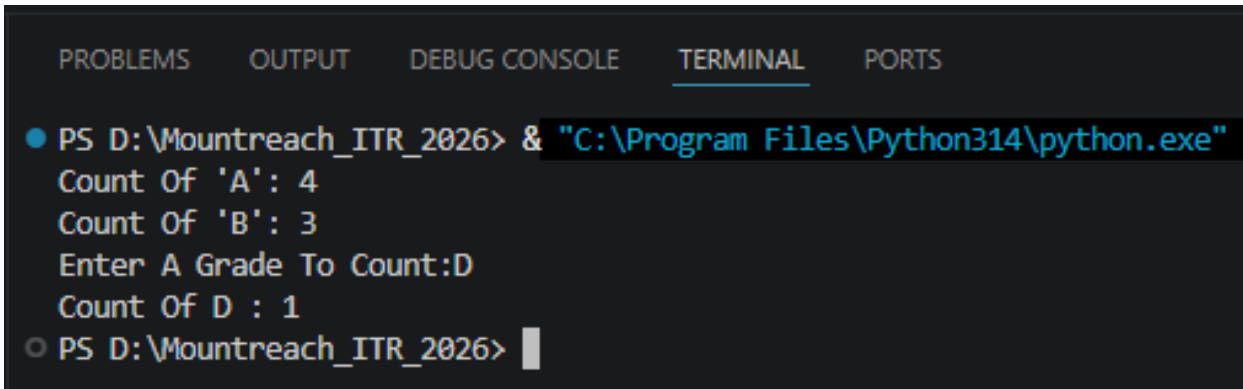
# Unpack Into Separate Variables
name, age, branch, grade = student
print("Name:", name)
print("Age:", age)
print("Branch:", branch)
print("Grade:", grade)
```

7. count() Method

Create the tuple: grades = ('A', 'B', 'A', 'C', 'A', 'B', 'D', 'A', 'B')

- Count and print how many times 'A' appears.
- Count and print how many times 'B' appears.
- Take a grade as input from the user and print how many times it appears.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
Count Of 'A': 4
Count Of 'B': 3
Enter A Grade To Count:D
Count Of D : 1
PS D:\Mountreach_ITR_2026>
```

Program Code:

```
grades = ('A', 'B', 'A', 'C', 'A', 'B', 'D', 'A', 'B')

print("Count Of 'A':", grades.count('A'))
print("Count Of 'B':", grades.count('B'))

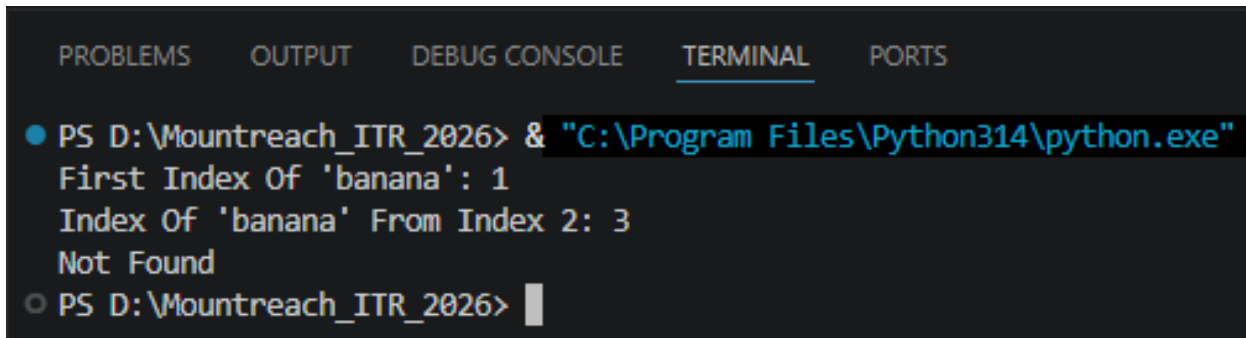
user_grade = input("Enter A Grade To Count:")
print("Count Of", user_grade, ":", grades.count(user_grade))
```

8. index() Method

Create the tuple: fruits = ('apple', 'banana', 'cherry', 'banana', 'mango', 'apple')

- Find and print the first index of 'banana'.
- Find and print the first index of 'banana' starting the search from index 2.
- Use try-except to safely find the index of 'kiwi' and print "Not found" if it doesn't exist.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
First Index Of 'banana': 1
Index Of 'banana' From Index 2: 3
Not Found
○ PS D:\Mountreach_ITR_2026> █
```

Program Code:

```
fruits = ('apple', 'banana', 'cherry', 'banana', 'mango', 'apple')

# First Occurrence Of 'banana'
print("First Index Of 'banana':", fruits.index('banana'))

# First 'banana' Starting From Index 2
print("Index Of 'banana' From Index 2:", fruits.index('banana', 2))

# Safe Search For 'kiwi'
try:
    print("Index Of 'kiwi':", fruits.index('kiwi'))
except ValueError:
    print("Not Found")
```

9. Immutability Of Tuples

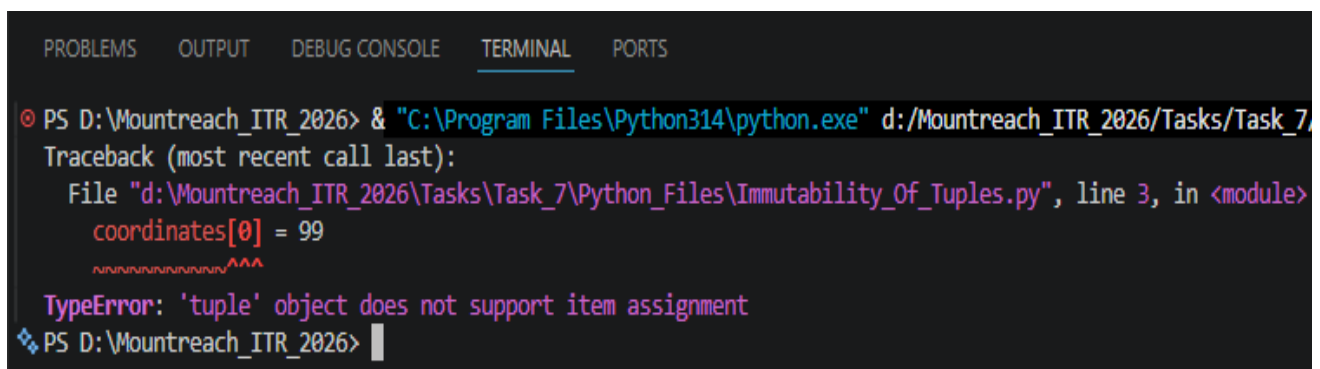
Create a tuple: `coordinates = (10, 20)`

Demonstrate that tuples are immutable by trying to:

- Change the first element
- Add a new element using `append()`

Explain what errors occur (in comments). Then show the correct way to modify data by converting the tuple to a list, making changes, and converting back to a tuple.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe" d:/Mountreach_ITR_2026/Tasks/Task_7/
Traceback (most recent call last):
  File "d:\Mountreach_ITR_2026\Tasks\Task_7\Python_Files\Immutability_Of_Tuples.py", line 3, in <module>
    coordinates[0] = 99
    ~~~~~^
TypeError: 'tuple' object does not support item assignment
PS D:\Mountreach_ITR_2026> |
```

Program Code:

```
coordinates = (10, 20)

coordinates[0] = 99
# This Will Raise A TypeError Because Tuples Are Immutable
# Trying To Change An Element - Raises TypeError:
# 'tuple' Object Does Not Support Item Assignment
# coordinates[0] = 99 <- This Would Crash

coordinates.append(30)
# This Will Raise An AttributeError Because Tuples Are Immutable
# Trying To Append - Raises AttributeError:
# 'tuple' Object Has No Attribute 'append'
# coordinates.append(30) <- This Would Crash

# Correct Way: Convert To List, Modify, Convert Back
temp = list(coordinates)
temp[0] = 99
temp.append(30)
coordinates = tuple(temp)
print("Modified Coordinates:", coordinates)
```

10. Mini Project - Combined Concepts

Create a **Student Record Program** using tuples:

- Take input for name, roll number, 3 subject marks.
- Store all information in a tuple (use packing).
- Print the complete record.
- Unpack the tuple and print each value with proper labels.
- Use count() to check how many times a particular mark appears (if any).
- Create a nested tuple to store multiple student records (at least 2 students) and access specific values.

Add meaningful comments and use proper formatting.

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
Enter Student Name: Mihir Patil
Enter Roll Number: 38
Enter Marks For Subject 1: 97
Enter Marks For Subject 2: 78
Enter Marks For Subject 3: 39

Complete Record: ('Mihir Patil', 38, 97, 78, 39)

Name: Mihir Patil
Roll No.: 38
Subject 1: 97
Subject 2: 78
Subject 3: 39

Enter A Mark To Check Frequency: 89
Mark 89 Appears 0 Time.

All Student Records
('Mihir', 'R01', 85, 90, 78)
('Kunal', 'R02', 92, 88, 95)

Second Student's Name: Kunal
First Student's Subject 2 Marks: 90
○ PS D:\Mountreach_ITR_2026> █
```

Program Code:

```
name = input("Enter Student Name: ")
roll = int(input("Enter Roll Number: "))
marks1 = int(input("Enter Marks For Subject 1: "))
marks2 = int(input("Enter Marks For Subject 2: "))
marks3 = int(input("Enter Marks For Subject 3: "))

# Store Using Tuple Packing
record = name, roll, marks1, marks2, marks3

# Print Complete Record
print("\nComplete Record:", record)

# Unpack & Display
s_name, s_roll, s_m1, s_m2, s_m3 = record
print("\nName:", s_name)
print("Roll No.:", s_roll)
print("Subject 1:", s_m1)
print("Subject 2:", s_m2)
print("Subject 3:", s_m3)

# count() Usage
marks_only = (s_m1, s_m2, s_m3)
check_mark = int(input("\nEnter A Mark To Check Frequency: "))
print("Mark", check_mark, "Appears", marks_only.count(check_mark), "Time.")

# Nested Tuple With Multiple Students
all_students = (("Mihir", "R01", 85, 90, 78), ("Kunal", "R02", 92, 88, 95))

print("\nAll Student Records")
for s in all_students:
    print(s)

# Accessing Specific Values
print("\nSecond Student's Name:", all_students[1][0])
print("First Student's Subject 2 Marks:", all_students[0][3])
```